Énoncé du Mini-Projet

Gestion d'une Clinique

Clinique SantéPlus

Architecture MVC Java EE · Base de données MySQL · Interface JSP/HTML/CSS

Table des matières

1 Présentation du sujet **2**

2 Description des besoins **2**

2.1 Les acteurs du système 2

2.2 Fonctionnalités attendues 2

2.3 Contraintes du système 3

3 Démarche de réalisation **4**

1. Présentation du sujet

Contexte du projet

La **Clinique SantéPlus** est une clinique médicale privée qui reçoit chaque jour des dizaines de patients. Aujourd'hui, toute la gestion est faite manuellement : les rendez-vous sont notés dans un agenda papier, les dossiers des patients sont des classeurs, et les médecins n'ont aucune vue synthétique de leur planning.
La direction souhaite développer une **application web** pour informatiser la gestion quotidienne de la clinique. Votre mission est de concevoir et de développer ce système, en appliquant les bonnes pratiques du génie logiciel.

2. Description des besoins

2.1 Les acteurs du système

Trois catégories d'utilisateurs vont interagir avec l'application :

- **L'administrateur** : il supervise l'ensemble du système. Il gère les comptes des utilisateurs (médecins, secrétaires), les spécialités médicales et peut consulter des statistiques globales de la clinique.
- **Le médecin** : il consulte son planning du jour et de la semaine, accède aux dossiers de ses patients et enregistre les résultats de ses consultations (diagnostic, prescription, paramètres physiologiques).
- **La secrétaire** : elle est le point d'entrée administratif. Elle enregistre les nouveaux patients, gère les rendez-vous (création, modification, annulation) et peut rechercher un patient dans le système.

2.2 Fonctionnalités attendues

Gestion des patients

- Enregistrer un nouveau patient avec ses informations personnelles (nom, prénom, date de naissance, téléphone, email, adresse, groupe sanguin).
- Modifier les informations d'un patient existant.
- Supprimer un patient (si aucun rendez-vous actif ne lui est associé).
- Rechercher un patient par nom, prénom ou email.
- Consulter la liste complète des patients.

Gestion des rendez-vous

- Créer un rendez-vous en associant un patient à un médecin sur un créneau horaire donné, avec indication du motif.
- Vérifier automatiquement la disponibilité du médecin avant de valider un créneau.
- Modifier ou annuler un rendez-vous existant.
- Afficher la liste des rendez-vous d'une journée ou d'une semaine, filtrée par médecin.
- Gérer les statuts d'un rendez-vous : *En attente*, *Confirmé*, *Annulé*, *Terminé*.

Gestion des consultations

- Permettre au médecin d'enregistrer une consultation à l'issue d'un rendez-vous : diagnostic, prescription, poids, tension artérielle, notes libres.
- Lier automatiquement la consultation au rendez-vous correspondant et passer ce dernier au statut *Terminé*.
- Consulter l'historique des consultations d'un patient (dossier médical simplifié).

Authentification et sécurité

- Chaque utilisateur (administrateur, médecin, secrétaire) doit s'authentifier avec un identifiant et un mot de passe.
- L'accès aux fonctionnalités est conditionné par le rôle de l'utilisateur connecté (contrôle d'accès).
- Une session est ouverte à la connexion et invalidée à la déconnexion.

2.3 Contraintes du système

- Un médecin ne peut pas avoir deux rendez-vous sur le même créneau.
- Un rendez-vous annulé ne peut pas être modifié.
- Un patient ne peut être supprimé que s'il n'a aucune consultation enregistrée.
- Les mots de passe ne doivent jamais être stockés en clair dans la base.

3. Démarche de réalisation

Ce projet se déroule en **quatre grandes étapes**, chacune correspondant à une phase du cycle de développement logiciel. Vous avancerez étape par étape, en validant chaque phase avant de passer à la suivante.

Étape 1 — Conception UML

La conception précède toujours le développement. Vous devez modéliser le système avec trois types de diagrammes UML :

- **Diagramme de cas d'utilisation** (*Use Case Diagram*) : identifiez tous les acteurs et toutes les fonctionnalités du système. Représentez les relations entre acteurs et cas d'utilisation (`«include»`, `«extend»`). *Outil conseillé : StarUML ou draw.io.*
- **Diagramme de classes** : identifiez les entités du domaine métier (Patient, Médecin, RendezVous, Consultation...), leurs attributs, leurs méthodes et les associations entre elles (multiplicités, navigabilité, héritage). *Ce diagramme sera la base directe de vos classes Java.*
- **Diagrammes de séquence** : pour au moins deux scénarios représentatifs (ex. : s'authentifier, prendre un rendez-vous), montrez les échanges entre les objets dans le temps (acteur → interface → contrôleur → base de données).

Étape 2 — Modélisation de la base de données

À partir du diagramme de classes, vous construirez le modèle de données :

- **MCD (Modèle Conceptuel de Données)** : représentez les entités et leurs associations avec les cardinalités, sans vous préoccuper encore des tables SQL.
- **MLD (Modèle Logique de Données)** : transformez le MCD en schéma relationnel en appliquant les règles de passage (clés primaires, clés étrangères, gestion des associations N,N).
- **Script SQL** : rédigez le script de création des tables sous MySQL (`CREATE TABLE`, contraintes `FOREIGN KEY`, `ENUM`...) et insérez quelques données de test (`INSERT INTO`).

Étape 3 — Développement Java EE (Architecture MVC)

L'implémentation suit strictement l'architecture **MVC** (Modèle – Vue – Contrôleur) :

- **Model (Modèle)** :
  - *Classes Java (POJO)* : une classe par entité du diagramme de classes (attributs privés + getters/setters + méthodes métier).
  - *Couche DAO (Data Access Object)* : une classe DAO par entité, responsable de toutes les interactions avec la base de données (connexion JDBC, requêtes SQL préparées, mapping ResultSet → objet).
- **View (Vue)** :
  - Pages *JSP* pour afficher les données (listes, formulaires, messages de retour). Les JSP ne contiennent **aucune logique métier**.
  - Mise en forme avec *HTML5 + CSS3*.
  - Utilisation des *JSTL* (`<c:forEach>`, `<c:if>`...) pour injecter les données.
- **Controller (Contrôleur)** :
  - Un *Servlet Java* par entité principale (ex. `PatientServlet`, `RdvServlet`...).
  - Le Servlet reçoit les requêtes HTTP, appelle les DAO, place les résultats dans la requête ou la session, puis redirige vers la JSP appropriée.

Étape 4 — Tests fonctionnels

Pour chaque fonctionnalité développée, vous rédigerez un **scénario de test** précisant : les données d'entrée, les actions à réaliser, et le résultat attendu. Vous vérifierez en particulier :

- Les cas nominaux (fonctionnement normal).
- Les cas limites (champs vides, créneau déjà pris, patient inexistant...).
- La cohérence des données en base après chaque opération.